ADR AI Agent — Handoff Document

ADR AI Agent — Handoff Document

Last Updated: 2026-05-18
Authors: Nathan (N459638)
Repos: [careconnect-adr_ai_agent](https://github.com/cvs-health-source-code/careconnect-adr_ai_agent) | [care-connect_member-adr-oci](https://github.com/cvs-legacy-source-code/care-connect_member-adr-oci)

---

1. Executive Summary

We built an **AI-powered assistant** embedded in CareConnect's Clinical Data Viewer (CDV) that lets clinical reviewers ask natural-language questions about ADR (Additional Document Request) documents. The assistant uses Vertex AI (Gemini) with retrieval-augmented generation (RAG) over patient-specific documents.

Where it runs: Python FastAPI app on GKE (`careconnect-ai-dev1` namespace), integrated into the existing Java OCI service via iframe.

Current state: Deployed to dev1/QA2 with multi-pod scaling (2-3 replicas), persistent sessions via Cloud SQL, and end-to-end tested through the Unqork QA environment.

Key capabilities:

- Multi-turn conversational AI grounded in uploaded patient documents
- Session persistence across pod restarts (PostgreSQL-backed checkpointer)
- Silent session retry — transparent to end users
- Document processing via Vertex AI Vision OCR
- Policy document search (clinical policy bulletins)
- Embedded widget with postMessage-based communication

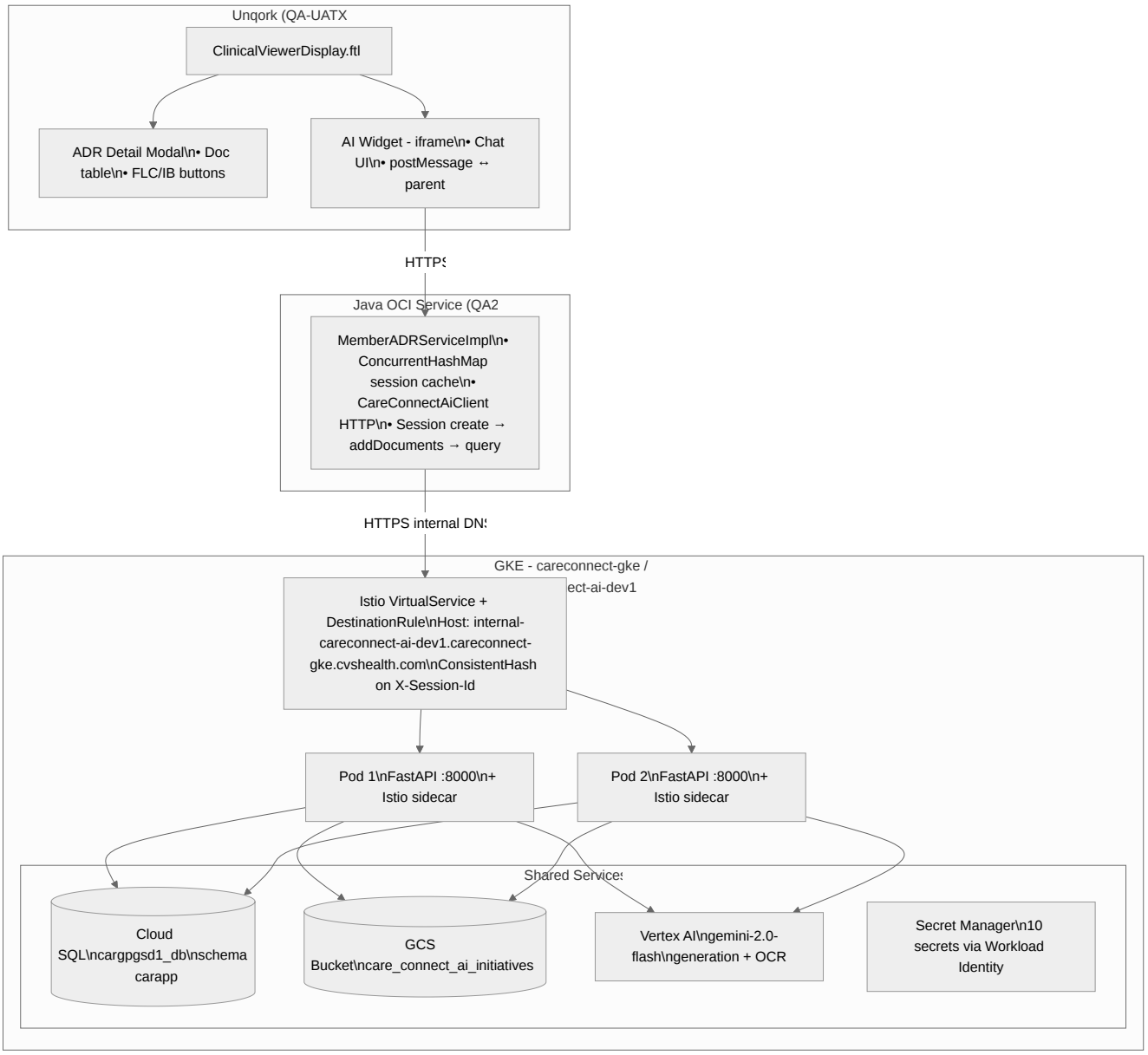
Team Ownership

Area	Owner	Contact
AI Agent (Python app)	Nathan / our team	This repo

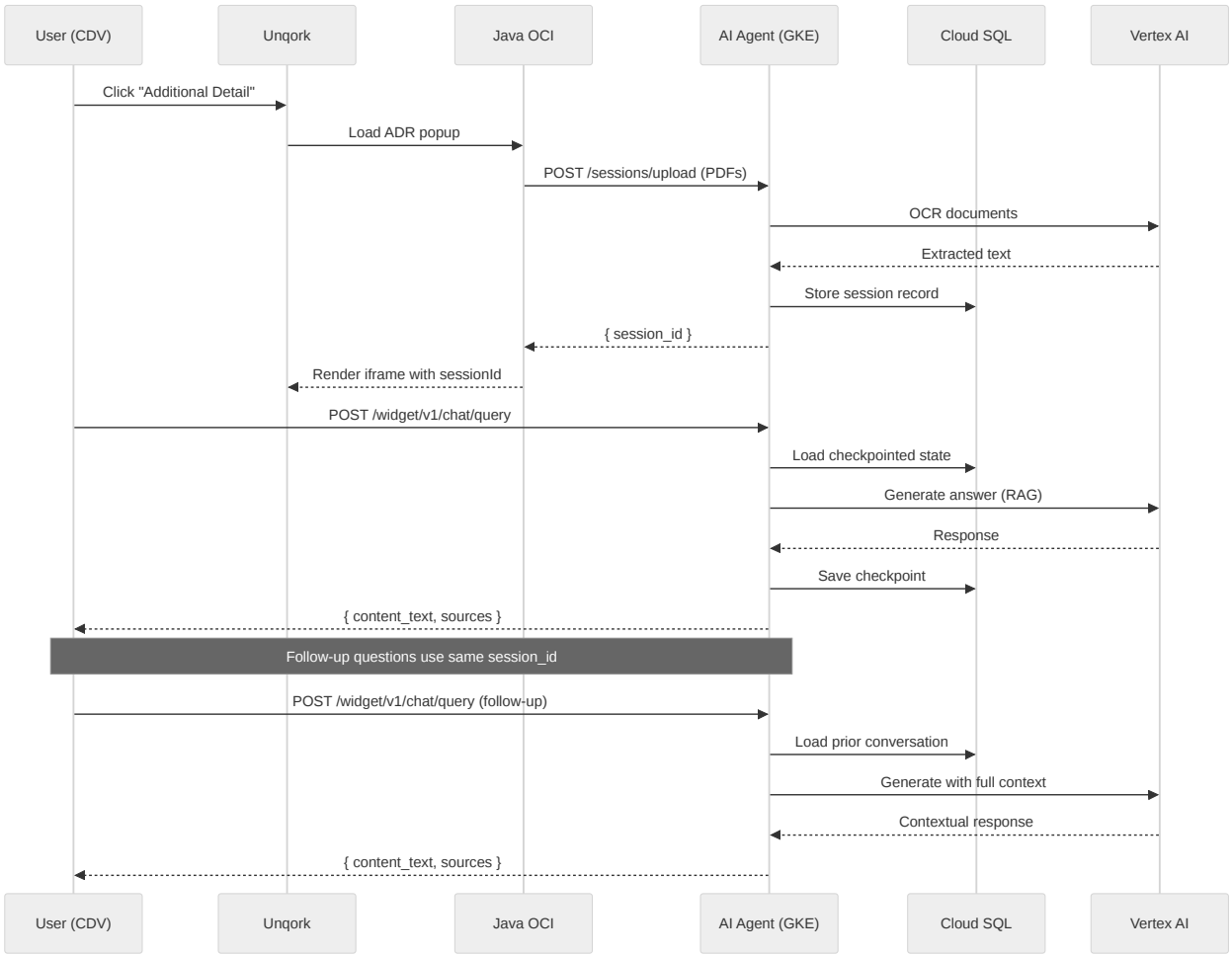
Java widget integration	Nathan / our team	care-connect_member-adr-oci
AI Engine (Gemini prompts, grounding)	Farhan's team	—
GKE infrastructure	Si / Leif	Slack
Unqork platform	CareConnect platform team	—
Cloud SQL, GCS, Secrets	Si / Leif (provisioned)	—

---

2. System Architecture



Data Flow (Sequence)



Data Flow

- **User clicks "Additional Detail"** in CDV (Unqork) → Java OCI service receives encounter context
- **Java creates AI session** → `POST /api/v1/sessions/upload` with PDF documents (bearer token auth)
- **AI Agent processes docs** → OCR via Vertex AI Vision → chunked → embedded in vector store (in-memory per session)
- **Widget iframe loads** → `GET /widget/v1/chat/ui?sessionId=X&layout=embedded`

- **User asks question** → widget JS `POST /widget/v1/chat/query` → agent invokes RAG pipeline → Gemini generates answer grounded in docs
- **Follow-up questions** → same session ID → checkpointer retrieves prior conversation from Cloud SQL → maintains context

Ownership Boundaries

- **We own:** All code in both repos, deployment config, CI/CD pipelines, widget UI
- **Farhan's team owns:** AI engine prompt design, grounding judge logic (embedded in our system prompt)
- **Si/Leif own:** GKE cluster, Cloud SQL instance, Secret Manager provisioning, DNS, Istio gateway certs
- **Unqork platform owns:** CDV module, Plug-In component framework, authentication flow

---

3. Feature Matrix

Feature	Status	Notes
Session creation (GCS URIs)	Done	`POST /api/v1/sessions`
Session creation (file upload)	Done	`POST /api/v1/sessions/upload` — multipart PDF
Incremental documents	Done	`POST /api/v1/sessions/{id}/documents`
Multi-turn query	Done	Maintains conversation history via checkpointer
Session persistence (Cloud SQL)	Done	AsyncPostgresSaver, survives pod restarts
Multi-pod scaling	Done	minReplicas=2, maxReplicas=3, Istio consistent hash
Silent session retry	Done	Transparent 2s retry on 404/500, no user message

Widget chat UI (iframe)	Done	Expand/collapse, scroll, height reporting
Widget styling	Done	Purple/white theme matching CareConnect
Source citations	Done	Page numbers extracted from RAG results
Session expiry cleanup	Done	24h TTL, hourly background task
DB session rehydration	Done	Falls back to DB lookup when not in memory
Policy document search	Done	Separate vector store for clinical policies
Feedback collection	Done	Thumbs up/down per response
Conversation history API	Done	`GET /api/v1/sessions/{id}/history`
Health endpoints	Done	`/health` (liveness) + `/health/ready` (readiness)
Java session cache	Done	ConcurrentHashMap, 500 max entries, LRU eviction
Java 404 retry	Done	Detects expired session, recreates from scratch
FreeMarker postMessage handler	Done	`careconnect:session-expired` → `buttonClick(null, "ADRRefresh")`
CI/CD (GHA → GAR → GKE)	Done	App Deployment workflow on push to non-prod
Stargate CD (proper release flow)	Pending	PR #335 merged, untested

FileNet integration	Not started	Meeting pending with FileNet team
Production deployment	Not started	Needs separate namespace, DNS, certs
Monitoring/alerting	Not started	No Grafana dashboards yet

---

4. Deployment & Operations Runbook

AI Agent Deploy (Current Pattern — Temporary)

The Stargate CD flow is not yet validated. We deploy by building locally and pushing to GAR:

```
cd ~/dev/git/cvshealth/careconnect-adr_ai_agent

<h1>1. Get the commit SHA</h1>
<p>IMAGE_TAG=$(git rev-parse HEAD)</p>

<h1>2. Build for linux/amd64</h1>
<p>docker build --platform linux/amd64 \</p>
<p>  -t us-east4-docker.pkg.dev/hcb-dev-careconnect-etl/cpcc-testrepo/careconnect-adr-ai-agent:$IMAGE_TAG .</p>

<h1>3. Push to GAR</h1>
<p>docker push us-east4-docker.pkg.dev/hcb-dev-careconnect-etl/cpcc-testrepo/careconnect-adr-ai-agent:$IMAGE_TAG</p>

<h1>4. Update deploy config</h1>
<h1>Edit deploy-configs/careconnect-ai-dev1/values/v1.yaml</h1>
<h1>Set dockerImageTag, currentAppVersion, canaryAppVersion to $IMAGE_TAG</h1>

<h1>5. Commit and push (triggers App Deployment workflow)</h1>
<p>git add deploy-configs/</p>
<p>git commit -m "chore(cd): deploy $IMAGE_TAG — <description>"</p>
<p>git push origin non-prod</p>
<p>
```

The `App Deployment` GHA workflow detects the v1.yaml change and applies it to GKE via Helm.

Java QA2 Deploy

```
# 1. Push code to release branch (auto-triggers build)
<p>cd ~/dev/git/cvshealth/care-connect_member-adr-oci</p>
<p>git push origin HEAD  # e.g., release/2026.06.R1</p>

<h1>2. Wait for build to complete (~20 min)</h1>
<p>gh run list --repo cvs-legacy-source-code/care-connect_member-adr-oci \</p>
<p>  --workflow "Build and Deploy to OCI" --limit 1</p>

<h1>3. Get artifact name from build logs</h1>
<p>gh run view <RUN_ID> --repo cvs-legacy-source-code/care-connect_member-adr-oci \</p>
```

```
<p>  --log 2>&1 | grep "PACKAGE_NAME"</p>

<h1>4. Dispatch deploy to QA2</h1>
<p>gh workflow run "Run Kubernates Deploy Reusable Workflow" \</p>
<p>  --repo cvs-legacy-source-code/care-connect_member-adr-oci \</p>
<p>  --ref master \</p>
<p>  -f environment=QA2 \</p>
<p>  -f artifact="member-adr-oci.P0_<sha>_<num>.tgz" \</p>
<p>  -f app_name=member-adr-oci</p>
<p>
```

Scaling

Edit `deploy-configs/careconnect-ai-dev1/values/v1.yaml`:

```
hpaSettings:
<p>  minReplicas: 2   # minimum pods always running</p>
<p>  maxReplicas: 3   # scales up under CPU load</p>
<p>  targetCPUUtilizationPercentage: 80</p>
<p>
```

Commit and push to `non-prod` — GHA applies the change.

Viewing Logs

```
# List pods
<p>kubectl get pods -n careconnect-ai-dev1</p>

<h1>Tail logs from a specific pod</h1>
<p>kubectl logs -n careconnect-ai-dev1 <POD_NAME> -c careconnect-adr-ai-agent --tail=100</p>

<h1>Search for errors</h1>
<p>kubectl logs -n careconnect-ai-dev1 <POD_NAME> -c careconnect-adr-ai-agent | grep -i
"error\|exception\|traceback"</p>

<h1>Startup logs (checkpointer init)</h1>
<p>kubectl logs -n careconnect-ai-dev1 <POD_NAME> -c careconnect-adr-ai-agent | grep -i
"agent\|checkpointer"</p>
<p>
```

Pod Restart

```
# Rolling restart (zero downtime with 2+ replicas)
<p>kubectl rollout restart deployment -n careconnect-ai-dev1</p>

<h1>Check rollout status</h1>
<p>kubectl rollout status deployment -n careconnect-ai-dev1 careconnect-adr-ai-agent</p>
<p>
```

Troubleshooting

Symptom	Cause	Fix
"Session not found" on query	Session not in memory or DB	Check if pod restarted; DB rehydration should handle this. If persistent, check Cloud SQL connectivity.
Widget shows error message	Query returned 500	Check pod logs for traceback. Common: Vertex AI quota, DB connection pool exhausted.
Java gets 404 on addDocuments	AI agent pod restarted, session lost	Java already handles this — recreates session. If persistent, check pod health.
No response from AI agent	Pod OOM killed	Check `kubectl describe pod` for OOMKilled. Increase memory limits in v1.yaml.
"Agent not initialized" RuntimeError	`init_agent()` failed at startup	Check startup logs. Usually Cloud SQL unreachable. Falls back to MemorySaver.

---

5. Integration Contracts

REST API Endpoints

Method	Path	Auth	Purpose
`GET`	`/health`	None	Liveness probe
`GET`	`/health/ready`	None	Readiness probe
`POST`	`/api/v1/sessions`	Bearer	Create session from GCS URIs
`POST`	`/api/v1/sessions/upload`	Bearer	Create session from uploaded PDFs
`POST`	`/api/v1/sessions/{id}/documents`	Bearer	Add docs to existing session

`POST`	`/api/v1/sessions/{id}/query`	Bearer	Query the agent
`GET`	`/api/v1/sessions/{id}/history`	Bearer	Get conversation history
`GET`	`/api/v1/sessions/{id}`	Bearer	Get session metadata
`DELETE`	`/api/v1/sessions/{id}`	Bearer	Delete session
`GET`	`/api/v1/sessions`	Bearer	List active sessions
`POST`	`/api/v1/sessions/{id}/feedback`	Bearer	Submit thumbs up/down
`GET`	`/widget/v1/chat/ui`	Query param	Widget HTML page
`POST`	`/widget/v1/chat/query`	Header	Widget query (same as API query)

Session Create — Request/Response

```
// POST /api/v1/sessions/upload
<p>// Content-Type: multipart/form-data</p>
<p>// Authorization: Bearer <token></p>
<p>// Body: files[] = PDF files</p>

<p>// Response 200:</p>
<p>{</p>
<p>  "session_id": "adr-20260518-3e1ba2f6",</p>
<p>  "status": "ready",</p>
<p>  "documents_processed": 2,</p>
<p>  "processing_time_ms": 3421,</p>
<p>  "created_at": "2026-05-18T20:55:00Z",</p>
<p>  "message_count": 0,</p>
<p>  "metadata": null</p>
<p>}</p>
<p>
```

Query — Request/Response

```
// POST /api/v1/sessions/{id}/query
<p>// Content-Type: application/json</p>
<p>// Authorization: Bearer <token></p>
<p>{</p>
<p>  "message": "Summarize the patient encounter"</p>
<p>}</p>
```

```
<p>// Response 200:</p>
<p>{</p>
<p>  "content_text": "The patient, Dagny M. Hass...",</p>
<p>  "content_html": "<p>The patient, <strong>Dagny M. Hass</strong>...</p>",</p>
<p>  "sources": [</p>
<p>    {"document": "EMR_Report.pdf", "page": 3}</p>
<p>  ]</p>
<p>}</p>
<p>
```

postMessage Protocol

The widget iframe communicates with the parent (Unqork/FreeMarker) via `window.postMessage`:

Event Type	Direction	Payload	Purpose
`careconnect:ready`	iframe → parent	`{ session_id, mode }`	Widget loaded and ready
`careconnect:height`	iframe → parent	`{ height }`	Resize iframe to content
`careconnect:session-expired`	iframe → parent	`{ session_id }`	Session lost, please recreate
`careconnect:scroll-top`	iframe → parent	`{}`	Scroll parent to top
`careconnect:close`	iframe → parent	`{}`	User wants to close widget

All messages include `source: "careconnect"` for filtering.

Widget URL Parameters

```
/widget/v1/chat/ui?sessionId=<id>&layout=embedded&token=<optional>
<p>
```

- `sessionId` (required): The AI session ID from session creation
- `layout`: `embedded` (no header, compact) or `standalone` (full page)
- `token`: Optional auth token (also accepted via `Authorization` header)

---

6. Configuration & Secrets

Environment Variables (from v1.yaml)

Variable	Value	Source
`APP_ENV`	`production`	Hardcoded
`LOG_LEVEL`	`INFO`	Hardcoded
`API_PORT`	`8000`	Hardcoded
`GCP_PROJECT_ID`	`hcb-dev-careconnect-etl`	Hardcoded
`GCS_BUCKET_NAME`	`care_connect_ai_initiatives`	Hardcoded
`AI_AGENT_PREFIX_GCS`	`adr_ai_agent`	Hardcoded
`API_AUTH_TOKEN`	`***`	Secret Manager
`CARECONNECT_WIDGET_TOKEN`	`***`	Secret Manager
`CLOUDSQL_HOST`	`10.252.173.101`	Secret Manager
`CLOUDSQL_PORT`	`5432`	Secret Manager
`CLOUDSQL_DATABASE`	`cargpgsd1_db`	Secret Manager
`CLOUDSQL_USER`	`cargpgsd1_nh_user`	Secret Manager
`CLOUDSQL_PASSWORD`	`***`	Secret Manager

Secret Manager (GCP Project: `hcb-dev-careconnect-etl`)

Secret Name	Purpose
`API_AUTH_TOKEN`	Bearer token for Java → AI Agent API calls
`CARECONNECT_WIDGET_TOKEN`	Bearer token for widget JS → AI Agent calls
`CLOUDSQL_HOST`	Cloud SQL private IP
`CLOUDSQL_PORT`	Cloud SQL port
`CLOUDSQL_DATABASE`	Database name

`CLOUDSQL_DATABASE_SCHEMA`	Schema name (`carapp`)
`CLOUDSQL_USER`	DB username
`CLOUDSQL_PASSWORD`	DB password
`member-adr-oci-careconnect-widget-token`	Token used by Java service (same as CARECONNECT_WIDGET_TOKEN)

Workload Identity

- **K8s Service Account:** `careconnect-adr-ai-agent`
- **GCP Service Account:** `careconnect-ai-features@hcb-dev-careconnect-etl.iam.gserviceaccount.com`
- **Roles:** Secret Manager Secret Accessor, Storage Object Viewer, Vertex AI User

Cloud SQL

- **Instance:** Private IP `10.252.173.101:5432`
- **Database:** `cargpgsd1_db`
- **Schema:** `carapp`
- **Tables:** `ai_sessions` , `checkpoints` , `checkpoint_blobs` , `checkpoint_writes` , `checkpoint_migrations`
- **Connection string format:** `postgresql://:@:/?options=-csearch_path%3Dcarapp`

Istio Configuration

- **Internal host:** `internal-careconnect-ai-dev1.careconnect-gke.cvshealth.com`
- **Load balancing:** Consistent hash on `X-Session-Id` header (sticky sessions)
- **Traffic split:** 100% prod / 0% canary (no canary active)

---

7. Known Issues & Tech Debt

Active Issues

- **Stargate CD bypass** — We deploy by building locally and pushing to GAR. The `CD (release)` workflow exists but hasn't been validated end-to-end. PR #335 merged the Stargate SOD gate approval, but we haven't tested a proper `[publish]` commit on a release branch. **Temporary pattern until validated.**

- **Grounding judge over-refusal** — The AI occasionally refuses questions it should answer. Meta-questions like "what were my previous questions?" get refused because the grounding judge only allows answers verifiable from uploaded documents. This is by design for safety, but the boundary could be tuned.
- **No structured DB health check** — `/health/ready` doesn't verify Cloud SQL connectivity. If the DB goes down, sessions will fall back to MemorySaver (in-memory only, lost on pod restart) silently.
- **v1.yaml contains hardcoded secrets** — The deployment values file has actual secret values embedded (currently needed for the Helm chart). These should migrate to external-secrets or Vault-injected references. Low risk since the repo is private and internal.
- **Test suite requires local Postgres** — 5 tests in `test_dependencies.py` attempt to connect to a `postgres` host. These fail locally without Docker Compose. Not blocking CI.

Technical Debt

- `langgraph-checkpoint-postgres` **API footgun** — The `PostgresSaver.from_conn_string()` method is a `@contextmanager` that must be used with `with`. We hit this bug and fixed it by switching to `AsyncPostgresSaver` with a persistent connection. Documented here for awareness if anyone touches checkpointer code.
- **In-memory vector store per session** — Each session builds its own vector store in memory. This doesn't persist across pod restarts. Documents are re-processable via the DB session record, but the vector embeddings are lost. Future: consider a shared vector store or embedding cache.
- **Java ConcurrentHashMap has no eviction on pod restart** — The Java service caches session IDs in memory. When the Java pod restarts, the cache is empty but the AI sessions still exist. The Java service handles this gracefully (creates new sessions), but it means extra session creation calls after a Java restart.

---

8. Remaining Work (Prioritized)

P1 — Near-term

- **Validate Stargate CD flow** — Try a proper release branch deploy using the `CD (release)` workflow. If it works, retire the local-build pattern.
- **FileNet bitstream API** — Replace EPP-sourced PDFs with direct FileNet document retrieval. Meeting pending with FileNet team to understand the bitstream API.

- **DB health check endpoint** — Add Cloud SQL connectivity check to `/health/ready` so Kubernetes knows when the pod can't persist sessions.

P2 — Before production

- **Production deployment** — Separate GKE namespace (e.g., `careconnect-ai-prod`), production DNS, TLS certs via Istio gateway, production Cloud SQL instance.
- **Load testing** — Verify 3-pod scaling under concurrent users. Test session persistence when pods scale up/down.
- **Monitoring & alerting** — Grafana dashboards for response latency, error rates, session counts. PagerDuty integration for pod health.

P3 — Future enhancements

- **Session analytics** — Track usage metrics (questions/session, avg response time, document types).
- **Streaming responses** — SSE-based streaming for real-time token output (infrastructure exists in `query_stream.py` but not wired to widget).
- **Multi-language support** — System prompt is English-only currently.
- **Vault integration** — Migrate from Secret Manager to Vault-backed external secrets (jwtVaultClusterScope config exists but GHA setup failed — Slack thread open with Si, low priority since Secret Manager works).

Infrastructure Items (Si/Leif)

- **DNS RITM3038180** — Submitted 2026-05-01 (ETA was 8pm that day). Internal DNS `internal-careconnect-ai-dev1.careconnect-gke.cvshealth.com` is live and resolving. Confirm no further action needed.
- **Vault GHA auth** — The `vault-k8s-jwt-auth` repo workflow failed to generate a PR for our namespace. Slack thread open with Si. Non-blocking since Secret Manager covers our needs.

Team Actions

- **Vishnu onboarding** — Mubeen connecting new hire Vishnu to Si/Leif the week of 2026-05-19. Use Section 9 below as the starting point.

---

9. Onboarding Guide

Prerequisites

- Python 3.10+ (app uses 3.10 in Docker)
- Docker Desktop (for local builds and `docker-compose`)
- `gcloud` CLI (authenticated to `hcb-dev-careconnect-etl`)
- `kubectl` (configured for `careconnect-gke` cluster)
- `gh` CLI (for GitHub Actions workflows)

Local Dev Setup

```
# Clone the repo
<p>git clone git@github.com:cvs-health-source-code/careconnect-adr_ai_agent.git</p>
<p>cd careconnect-adr_ai_agent</p>

<h1>Create virtual environment</h1>
<p>python -m venv .venv && source .venv/bin/activate</p>

<h1>Install dependencies</h1>
<p>pip install -e ".[dev]"</p>

<h1>Set minimal env vars for local testing</h1>
<p>export APP_ENV=development</p>
<p>export LOG_LEVEL=DEBUG</p>
<p>export API_AUTH_TOKEN=dev-token-123</p>
<p>export GCP_PROJECT_ID=hcb-dev-careconnect-etl</p>

<h1>Run the app locally (no DB, uses MemorySaver)</h1>
<p>uvicorn src.api.app:create_app --factory --reload --port 8000</p>

<h1>Open widget: http://localhost:8000/widget/v1/chat/ui?
sessionId=test&layout=standalone</h1>
<p>
```

Running Tests

```
# Full unit test suite (~1000 tests, ~90s)
<p>pytest tests/unit/ -v</p>

<h1>Specific module</h1>
<p>pytest tests/unit/session_manager/ -v</p>

<h1>With coverage</h1>
<p>pytest tests/unit/ --cov=src --cov-report=html</p>
<p>
```

5 tests in `test_dependencies.py` require a local Postgres — these are expected to fail without Docker Compose.

How to Deploy

See Section 4 above. The TL;DR:

- Make changes, commit to `non-prod`
- Build Docker image: `docker build --platform linux/amd64 -t : .`
- Push: `docker push :`
- Update `v1.yaml` with new SHA
- Commit + push → GHA deploys automatically

Key Files to Read First

File	Purpose
`src/agents/graph.py`	LangGraph agent definition, checkpointer creation, invoke/stream helpers
`src/session_manager/core/agent_factory.py`	Singleton agent lifecycle (init, get, reset)
`src/api/templates/widget_chat.html`	The entire chat widget UI (HTML + JS in one file)
`src/api/routes/sessions.py`	Session CRUD endpoints
`src/api/routes/widget.py`	Widget-specific routes
`src/api/dependencies.py`	Auth, session lookup, registry
`src/api/app.py`	FastAPI app factory, lifespan (agent init, cleanup)
`deploy-configs/careconnect-ai-dev1/values/v1.yaml`	Full deployment configuration

For the Java side:

File	Purpose
`CareConnectAiClient.java`	HTTP client for AI Agent API
`MemberADRServiceImpl.java`	Session cache, retry logic

`ClinicalViewerDisplay.ftl`	FreeMarker template with iframe + postMessage handler

Who to Ask

Question	Person
GKE, Cloud SQL, DNS, secrets	Si / Leif
AI engine behavior, prompt tuning	Farhan
GKE cluster config, Istio	Rushminder / Toji
Unqork CDV module, Plug-In components	CareConnect platform team
Everything else (agent, widget, CI/CD)	Nathan (or this document)